

Marine Race Arena: A Configurable Simulation Environment for Underwater Multi-Robot Racing and Evaluation

Andrea Bedei
University of Bologna
Cesena, Italy
email: to be added

Abstract—Autonomous racing has become an effective way to stress perception, planning, control, and decision-making under reproducible constraints. Ground platforms such as F1TENTH and gate-based aerial racing have shown that competition formats can accelerate robotics research by standardizing tracks, rules, baselines, and evaluation metrics. In marine robotics, however, existing simulators and competitions mainly target inspection, navigation, intervention, or surface-vessel missions. They do not provide a dedicated, configurable underwater racing benchmark in which underwater vehicles traverse gate sequences under currents, obstacles, limited sensing, multi-robot interaction, and an explicit independent referee. This paper presents Marine Race Arena, a simulation-oriented evaluation layer for underwater racing. The system is designed around declarative race configurations, geometric gate definitions, acoustic beacon guidance, configurable marine disturbances, replaceable participant controllers, simulator adapters, event logging, scoring, and referee logic that remains separated from the autonomy stack under test. Rather than proposing a new controller or a new underwater physics simulator, Marine Race Arena provides a reproducible benchmark structure in which classical, vision-based, learning-based, manual, and oracle controllers can be compared under common rules. The current draft reports the architecture, benchmark protocol, and implemented baselines, while quantitative controller performance under currents remains part of ongoing work.

Index Terms—underwater robotics, autonomous racing, simulation benchmark, multi-robot systems, marine robotics, evaluation protocol, HoloOcean

I. INTRODUCTION

Evaluating autonomy for underwater vehicles is difficult for reasons that are both physical and methodological. The underwater domain combines six-degree-of-freedom motion, buoyancy, drag, limited visibility, noisy or intermittent sensing, delayed or low-bandwidth communication, and disturbances such as spatially varying currents. Real deployments also require tanks, pools, basins, boats, safety procedures, waterproof hardware, and repeated calibration. As a result, early-stage autonomy algorithms are often evaluated either in isolated simulation scenarios or in real experiments that are expensive to reproduce.

Autonomous racing offers a complementary evaluation paradigm. A race is not merely a demonstration of speed: it is a compact stress test for perception, state estimation, planning, control, recovery behavior, and rule compliance. The appeal

of platforms such as F1TENTH is that they convert autonomy into a repeatable competition problem with common tracks, common constraints, and quantitative metrics. A similar idea is visible in autonomous drone racing, where gate sequences, onboard sensing, aggressive motion, and timing rules create a clear benchmark for embodied intelligence.

The underwater case is less mature. Marine robotics already has strong simulators and competitions, but the dominant tasks are mission-oriented: inspection, localization, docking, manipulation, object search, mapping, surface-vessel navigation, and heterogeneous swarm missions. These are important, but they do not directly provide a repeatable underwater race format with gate sequences, marine disturbances, multi-vehicle interaction, swappable controller submissions, and an explicit referee independent of the controller.

This paper presents Marine Race Arena, a configurable simulation environment for underwater multi-robot racing and evaluation. The aim is not to introduce a new underwater vehicle simulator or to claim a validated sim-to-real racing platform. Instead, Marine Race Arena defines a benchmark layer above a simulator backend: tracks, gates, beacons, currents, obstacles, participants, controller interfaces, referee rules, scoring, and logs are specified as explicit configuration objects. Controllers receive observations and return commands, while the referee uses ground-truth simulator state only for evaluation. This separation mirrors the structure of serious benchmarks: the autonomy stack under test is isolated from the scoring system.

The contributions of this paper are:

- a configurable underwater race format based on gate sequences, laps, start/finish timing, acoustic beacons, marine currents, and obstacles;
- a modular architecture separating arena definition, simulator adapters, participants, controller plug-ins, event logging, scoring, and referee logic;
- an evaluation protocol with explicit benchmark tasks, penalties, completion metrics, and status handling for reproducible comparison;
- initial official baselines and a transparent implementation status that distinguishes implemented infrastructure from controller-performance claims still under evaluation.

The rest of the paper is organized as follows. Section II reviews autonomous racing, drone racing, marine simulators, marine competitions, and automated evaluation frameworks. Section III describes the system architecture. Section IV details arena design. Section V defines the evaluation protocol. Section VI summarizes the current implementation and baseline status. Sections VII and VIII discuss limitations and conclude.

II. RELATED WORK

A. Autonomous Racing as an Evaluation Paradigm

Autonomous racing has become a useful benchmark because it compresses several hard autonomy problems into a compact, repeatable task. Vehicles must perceive the environment, estimate state, plan dynamically feasible trajectories, react to disturbances or competitors, and execute commands under strict latency constraints. The F1/10 platform was introduced as an open-source, affordable, 1/10-scale autonomous vehicle testbed with sensors, software, simulation, and competitions intended to make cyber-physical autonomy research more reproducible and accessible [1]. Later surveys emphasize that F1TENTH/RoboRacer has grown into a broad research ecosystem, but also that the diversity of methods can make direct comparison difficult without unified benchmark definitions and baseline results [2], [3].

This history is important for underwater racing. The value of F1TENTH is not only the vehicle model; it is the combination of shared platform, software interfaces, competition rules, baselines, and metrics. Marine Race Arena follows this evaluation-centered view, but replaces the ground-track assumptions with underwater-specific elements: 3D gate traversal, depth regulation, acoustic-like guidance, marine currents, limited visibility, obstacles, and possible multi-robot interaction.

B. Gate-Based Autonomous Drone Racing

Aerial drone racing provides a second relevant model because the task is naturally gate-based and three-dimensional. The AlphaPilot system combined learned visual abstraction, nonlinear filtering, and time-optimal planning to navigate race gates during the 2019 autonomous drone racing world championship [4]. More recent systems have demonstrated champion-level autonomous drone racing using deep reinforcement learning [5], robust monocular perception and control in A2RL-style competition settings [6], and pro-level racing in uninstrumented arenas [7]. Other competition reports show the importance of drift correction, gate detection, and perception-aware planning when only limited onboard sensing is allowed [8].

Marine Race Arena borrows the structural idea of a gate sequence, not the vehicle dynamics or sensing assumptions. Aerial racing gates are usually crossed at high speed in air, often with lightweight cameras and IMUs. Underwater racing introduces different constraints: lower speed, stronger fluid disturbances relative to vehicle authority, depth safety, acoustic aids, light attenuation, and the possibility of evaluating ROV-like vehicles rather than only fully untethered AUVs. This

motivates a benchmark format that is similar in logic but different in physical assumptions.

C. Marine Robotics Simulators

A number of underwater simulators already address the need for realistic, repeatable marine robotics development. DAVE provides an open-source underwater simulation stack with vehicles, sensors, dynamic bathymetry, ocean currents, and tools for algorithm development [9]. Stonefish is an open-source marine robotics simulator with advanced hydrodynamics, sensor simulation, and recent extensions supporting machine-learning research through improved sensors, tethered operations, thruster modeling, visual-light communication, and annotation tools [10]. UNav-Sim focuses on visually realistic underwater simulation and synthetic data generation using Unreal Engine 5 [11]. Recent surveys compare HoloOcean, Stonefish, DAVE, MARUS, and UNav-Sim in terms of sensor fidelity, physical modeling, task support, and sim-to-real issues [12]. HoloOcean itself has also been extended toward higher-fidelity dynamics, Unreal Engine 5 rendering, ROS 2 support, sonar improvements, semantic sensors, and environment-generation tools [13].

These simulators are complementary to Marine Race Arena. They provide physics, rendering, vehicles, and sensors; Marine Race Arena provides the race-level abstraction: configured tracks, gate geometry, beacons, benchmark tasks, participant interfaces, referee state, penalties, ranking, and output logs. In other words, the proposed contribution is not another simulator backend but a benchmark and competition layer that can use such backends.

D. Marine Robotics Competitions and Maritime Benchmarks

Marine competitions have played an important role in robotics education and research. RoboSub challenges teams to build autonomous underwater vehicles capable of solving underwater mission tasks, and recent technical design reports illustrate how teams integrate vision, sonar, controls, and mechanical subsystems for the competition [14], [15]. Maritime RobotX focuses primarily on autonomous surface-vessel capabilities, while related work has explored how ideas from the AI Driving Olympics and Duckietown could be adapted to RobotX using containerized agents and uniform observation/action interfaces [16]. MBZIRC and other field robotics challenges show the importance of autonomous multi-robot mission execution, GPS-denied navigation, and rapid deployment under real-world conditions [17].

These competitions are valuable, but their objectives differ from underwater racing. RoboSub is mission-task oriented; RobotX is surface-vessel oriented; MBZIRC emphasizes heterogeneous aerial, ground, or maritime mission execution rather than structured underwater gate traversal. Marine Race Arena targets a narrower and more repeatable niche: underwater vehicles traversing race tracks under standardized timing and penalty rules.

E. Automated Evaluation and Plug-in Agent Interfaces

Modern autonomy benchmarks increasingly separate the evaluated agent from the evaluator. CARLA was introduced as an open urban driving simulator with configurable environments, sensors, and benchmark-style metrics [18]. The AI Driving Olympics used Duckietown tasks, simulators, code templates, baseline implementations, and standardized hardware to evaluate mobile-robot agents in both simulation and real settings [19]. Robotics extensions of OpenAI Gym further illustrate the value of common interfaces for comparing algorithms under the same virtual conditions [20].

Marine Race Arena adopts the same principle. A controller is a plug-in that receives observations and returns either high-level commands or lower-level thruster commands. The referee, in contrast, is not part of the controller. It uses privileged simulator state only to score the run, validate gate passages, detect penalties, and produce logs. This separation is essential: a controller may be acoustic-only, vision-assisted, learning-based, manual, or oracle-like for debugging, but all controllers are judged by the same independent protocol.

F. Positioning of Marine Race Arena

Table I summarizes the positioning. Existing work provides either racing platforms, gate-based aerial racing systems, underwater simulators, maritime competitions, or benchmark infrastructure. Marine Race Arena is designed to cover the specific intersection that remains weakly represented: configurable underwater racing with gate traversal, currents, obstacles, multi-robot support, independent scoring, and swappable controller plug-ins.

III. SYSTEM OVERVIEW

A. Design Principles

Marine Race Arena is built around four design principles. First, the arena must be declarative: a race should be described by configuration rather than hard-coded scenario logic. Second, the autonomy stack must be replaceable: the same race should accept manual, rule-based, learning-based, acoustic-only, vision-assisted, or debugging controllers through a common interface. Third, the referee must be independent from the controller: privileged information can be used for scoring, but it must not be exposed to official controllers. Fourth, the system should remain simulator-adapter based: the benchmark layer should not depend on a single rendering or dynamics implementation.

B. Architecture

Fig. 1 shows the high-level architecture. A race configuration defines the world bounds, gates, start pose, finish rule, beacon behavior, current profiles, obstacle generation, participants, controller selection, and referee settings. The arena builder converts this configuration into simulator objects and abstract race geometry. The simulator adapter steps the back-end and returns observations. Participant controllers consume observations and output commands. The referee consumes

simulator state, validates race progress, applies penalties, and writes logs and summaries.

C. Controller Interface

The controller interface follows a simple reset-step-close lifecycle. At reset time, a controller receives race metadata. At each step, it receives an observation containing allowed sensor values, beacon information, and race progress. It returns either a high-level body-frame command, for example surge, sway, heave, and yaw, or a lower-level thruster command. This interface is intentionally small: it makes it easy to compare controllers implemented with different assumptions and different internal software stacks.

Official controllers are expected to set two semantic flags: whether they are debug-only and whether they consume ground truth. In official evaluation, debug-only or ground-truth-consuming controllers should be excluded from scientific baselines. This allows oracle controllers to remain useful for debugging tracks and referee logic without contaminating benchmark claims.

D. Simulator Adapters

The current implementation distinguishes race logic from simulator execution. A fallback adapter supports simulator-independent tests of gate validation, scoring, logging, and command flow. A HoloOcean-oriented adapter targets underwater vehicle execution in a 3D marine simulator. This separation is important because the proposed benchmark should not be tied to one version of one simulator. It also allows the referee and configuration logic to be tested without requiring graphics, hydrodynamics, or real-time rendering.

IV. ENVIRONMENT DESIGN

A. Track Geometry and Gate Sequence

A race is represented as an ordered sequence of gates. Each gate has an identifier, position, orientation, internal aperture size, bar thickness, color, and passage direction. The referee validates the abstract gate geometry rather than relying on visual collision geometry. This distinction matters because the visual bars shown in a simulator are rendering objects, whereas the scoring rule must be deterministic and independent of visual artifacts.

The current reference format uses a two-lap race over an 11-gate sequence, for 22 valid crossings in total. The gate aperture is configurable; the current marine reference configuration uses a square internal opening of $1.5\text{ m} \times 1.5\text{ m}$. The system supports single gates, double gates, vertically separated gate pairs, and split-S-like underwater sequences in which the vehicle must change depth between consecutive gates. These structures adapt the gate logic of aerial racing to slower, disturbance-prone underwater motion.

B. Acoustic Beacon Guidance

Marine Race Arena includes acoustic-beacon-like guidance associated with the next expected gate. The beacon does not

TABLE I
POSITIONING OF MARINE RACE ARENA WITH RESPECT TO RELATED BENCHMARK FAMILIES. THE TABLE IS QUALITATIVE AND EMPHASIZES EVALUATION STRUCTURE RATHER THAN SIMULATOR FIDELITY ALONE.

Family	Domain	Race / gates	Swappable agents	Main limitation for this paper's target
F1TENTH/RoboRacer [1]–[3]	Ground vehicles	Yes, track racing	Yes	Ground dynamics and 2D road assumptions
Autonomous drone racing [4]–[6]	Aerial vehicles	Yes, 3D gates	Usually system-specific	Air dynamics, no marine currents or underwater sensing
Marine simulators [9]–[12]	Underwater / marine	Not primarily	Simulator-dependent	Provide physics and sensors, not a race referee and scoring format
RoboSub / RobotX / MBZIRC [14]–[17]	Marine robotics	Mission-oriented	Competition-dependent	Focus on task completion or surface vessels rather than repeatable underwater racing
Marine Race Arena	Underwater vehicles	Yes, configurable tracks	gate Yes	Current work is simulation-only and still requires quantitative baseline completion

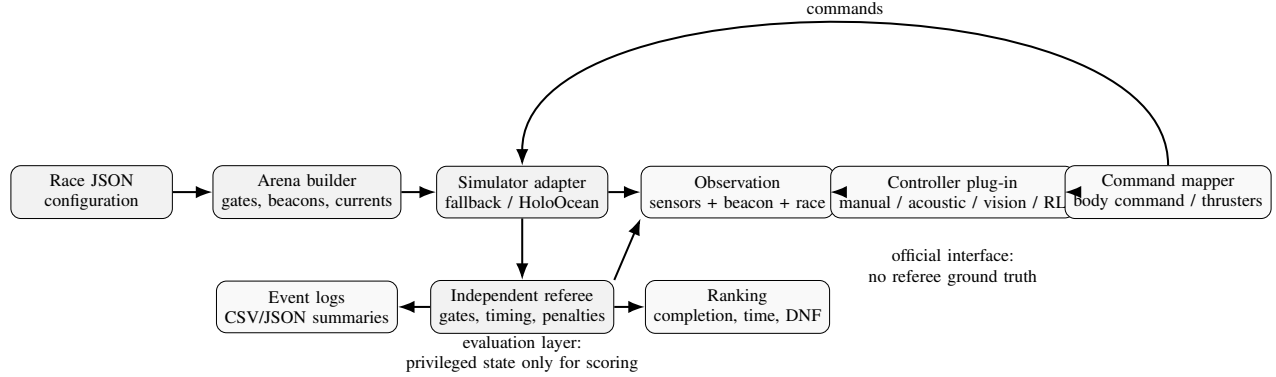


Fig. 1. Marine Race Arena architecture. The controller is a plug-in evaluated by an independent referee. Privileged simulator state is used for scoring and logging, but official controllers consume only allowed observations.

validate gate passage. Instead, it provides a restricted navigation cue such as bearing, elevation, range, signal strength, update rate, and optional noise/dropout parameters. This design allows an official controller to receive useful guidance without receiving exact privileged gate geometry.

This distinction is central to the benchmark. A controller may use the beacon to approach the next gate, but it still must regulate heading, depth, and lateral alignment under vehicle dynamics and disturbances. A vision-assisted controller may combine beacon guidance with local gate observations, while an acoustic-only baseline can be evaluated as a minimal non-cheating reference.

C. Marine Currents

Currents are modeled as configurable disturbances rather than hidden implementation details. The current configuration supports profiles such as constant flow, localized jets, sinusoidal fields, and vortex-like disturbances. This enables benchmark tasks in which the same track can be evaluated under no-current, moderate-current, and structured-current conditions.

The most important design choice is that currents belong to the environment configuration, not to the controller. A benchmark run should therefore record the active current profile, random seeds, obstacle settings, and simulator adapter in its metadata. This makes later comparisons meaningful: a controller that completes a clean track cannot be compared

directly with one evaluated under a localized jet unless the task definition records that difference.

D. Obstacles and Arena Bounds

The arena can include fixed or generated obstacles. Obstacles are defined with position, size, orientation, collision flag, penalty, and optionally the gate corridor in which they are placed. Generated obstacles are seeded so that obstacle-gate tasks can be repeated exactly. World bounds define legal operating space, including depth limits. Out-of-bounds events are logged and penalized according to referee settings.

The environment design therefore separates three types of geometry: abstract gate geometry for scoring, visual geometry for simulator rendering, and obstacle/bounds geometry for safety and penalties. This prevents the benchmark from depending on accidental simulator collisions for gate validation.

V. EVALUATION PROTOCOL

A. Gate Validation

A valid gate crossing occurs when the vehicle trajectory segment between two consecutive poses intersects the expected gate plane, the crossing direction matches the configured passage direction, and the intersection point lies inside the internal aperture after applying any configured clearance margin. If the vehicle crosses another gate while the referee expects a different gate, the event is treated as a missed-gate attempt and

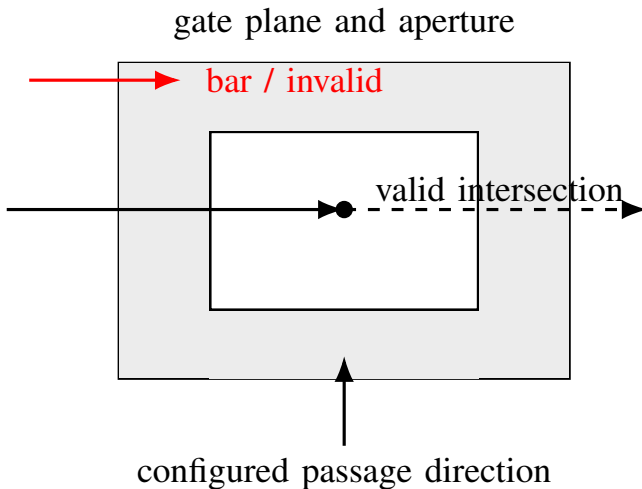


Fig. 2. Gate validation is performed on abstract gate geometry. The segment between previous and current vehicle positions must cross the expected gate plane in the correct direction and inside the aperture.

may trigger a penalty or DNF depending on the benchmark settings.

Fig. 2 illustrates the validation concept. The referee evaluates the vehicle center point in the current implementation. The interface is intentionally compatible with future full-body or footprint-based validation, which would shrink the valid aperture based on vehicle dimensions or explicit collision geometry.

B. Timing, Penalties, and Ranking

The protocol supports two timing modes. In green-to-finish timing, the official clock starts at the race start event and ends when the finish condition is satisfied. In first-gate-to-last-gate timing, the official clock starts at the first valid crossing of the first gate and ends at the final valid crossing of the finish gate. The latter is useful when one wants to exclude spawn settling time or manual launch transients from the official race time.

Penalties are accumulated as seconds added to the official time. Events include minor collision, gate collision, obstacle collision, out-of-bounds, stuck episode, missed gate, timeout, manual stop, and controller error. Ranking first prioritizes finish status and completed gates, then penalized time. The event log records all relevant events with timestamps so that a run can be audited after completion.

C. Benchmark Tasks

Table II defines the benchmark tasks used in the current design. The tasks are cumulative: clean-gate isolates navigation and gate sequencing; obstacle-gate adds spatial avoidance; current-gate adds marine disturbances; multi-ROV adds interaction and congestion effects.

D. Reported Metrics

For each controller and task, the benchmark runner should report completion rate, number of completed gates, official

TABLE II
BENCHMARK TASK DEFINITIONS.

Task	Purpose
<code>clean_gate</code>	One vehicle, gate sequence, no active obstacles, no active currents. Tests basic navigation, depth regulation, and referee consistency.
<code>obstacle_gate</code>	Gate sequence with fixed or seeded generated obstacles. Tests local avoidance and robustness to clutter.
<code>current_gate</code>	Gate sequence under configured current fields. Tests controller robustness to marine disturbances.
<code>multi_rov</code>	Multiple participants in the same arena. Tests referee scalability, ranking, interaction handling, and future multi-agent policies.

time, penalized time, collisions, obstacle collisions, out-of-bounds events, stuck events, DNF reasons, manual stops, controller errors, and per-seed summaries. These metrics deliberately combine performance and reliability. A controller that is fast but frequently misses gates should not be ranked as more robust than a slower controller with higher completion rate.

VI. IMPLEMENTATION STATUS AND BASELINES

A. Current Software Implementation

The current software implementation follows the architecture described above and is maintained in the public Marine Race Arena repository [21]. The repository contains configuration dataclasses, race validation, arena components, simulator adapters, participant interfaces, controller loading, baseline controllers, referee state, scoring, logging, benchmark scripts, and simulator-independent tests. The implementation also includes reference tracks and command-line entry points for validating track configurations and running repeated benchmark trials.

The benchmark runner is designed to execute the same track over multiple seeds and produce per-run logs plus aggregate CSV/JSON summaries. This is important for paper-quality evaluation because single runs are not sufficient in a stochastic or disturbance-rich environment. Even deterministic controllers should be evaluated over changes in obstacle generation, current profiles, sensor noise, or initial perturbations.

B. Official Baselines

The repository currently defines two main non-cheating official baselines. The `acoustic_baseline` is deterministic and uses beacon, sensor, and race observations. It implements an approach/transit/exit gate state machine, yaw smoothing, bearing deadbands, vertical regulation, and speed scheduling. The `rule_gate_baseline` combines beacon guidance with front-camera gate information when available. It uses local visual centering for yaw, sway, and heave, and reduces or stops forward surge until alignment is acceptable. Experimental or deprecated controllers, such as acoustic-vision and vision-gate variants, remain useful for debugging but should not be treated as final scientific baselines until their stability is quantified.

TABLE III
CURRENT IMPLEMENTATION AND EVIDENCE STATUS.

Component		Current status	Quantitative evidence
Configurable tracks	race	Declarative track files	Structural validation
Gate referee		Center-point aperture crossing	Unit validation
Scoring and logs		Event logs and summaries	Needs benchmark runs
Acoustic baseline		Official non-cheating controller	Needs task table
Rule gate baseline		Implemented; robust transit tuning	Needs task table
Current-gate task		Defined in configuration	Tuning ongoing
Multi-ROV task		Future-ready task/interface	No final experiment
Real-world validation		Not available in this draft	Not claimed

The key point is that these baselines are reference controllers, not the main contribution. Their role is to prove that the arena can evaluate different autonomy stacks under common rules. The final paper should eventually include completion rates and penalized times for at least acoustic-only and rule-based vision-assisted baselines on clean-gate, obstacle-gate, and current-gate tasks.

C. What Is Not Yet Claimed

This draft intentionally does not report numerical controller performance. At the time of writing, the current-gate baseline with marine currents is still under development, and no full benchmark table has been finalized. Table III records the current state without fabricating results.

D. Recommended Final Experimental Matrix

The final experimental section should contain, at minimum, three controllers and three benchmark tasks. The controllers should be: an acoustic-only baseline, the rule-based gate baseline, and an oracle/debug controller used only as an upper-bound track sanity check. The tasks should be clean-gate, obstacle-gate, and current-gate. For scientific reporting, each controller-task pair should be executed over multiple seeds, with fixed random seeds recorded in metadata. The oracle should be excluded from official ranking tables or clearly marked as debug-only.

A complete result table should include completion rate, mean completed gates, mean official time for completed runs, mean penalized time, collisions, out-of-bounds events, stuck events, and DNF reasons. Only after this table is produced should the abstract and conclusion contain performance claims.

VII. DISCUSSION

Marine Race Arena is best understood as evaluation infrastructure rather than a controller paper. This distinction is important. A new controller can be impressive but difficult to compare if every paper uses different scenarios, metrics, and hidden assumptions. A benchmark arena has value when it allows many controllers to be tested under the same rules.

The proposed design therefore emphasizes explicit configurations, official sensor restrictions, repeatable tasks, and referee-controlled scoring.

The main limitation is that the current paper is simulation-only. This is not a minor limitation: underwater sim-to-real transfer is hard because visual appearance, hydrodynamics, tether effects, actuation limits, acoustic propagation, lighting, and sensor noise can differ substantially between simulation and water-tank experiments. For this reason, the current contribution should not be framed as a validated predictor of real underwater racing performance. It should be framed as a reproducible development and evaluation layer that can later be connected to physical tests.

A second limitation concerns gate validation. The current center-point rule is deterministic and simple, but it is not a full vehicle-body clearance model. A vehicle can have a valid center crossing while still passing too close to a physical bar. Clearance margins partially address this, but future versions should support footprint- or mesh-aware validation.

A third limitation is controller maturity. The acoustic baseline and rule-based vision baseline are useful starting points, but the final paper should include systematic runs before claiming robustness. In particular, currents are the decisive underwater contribution: if the arena includes currents but the baselines are reported only on clean tracks, the evaluation will look incomplete. The strongest final version of the paper should therefore include current-gate results, even if the baseline fails often. Failure under currents is still valuable if it exposes why the benchmark is needed.

Future work should extend the arena in four directions: richer hydrodynamic profiles, sensor-degradation models, full-body gate validation, and standardized submission containers for external controllers. A later sim-to-real extension could use a small tank and a BlueROV2-style platform to validate whether the same referee and controller interface can support real-water trials.

VIII. CONCLUSION

This paper presented Marine Race Arena, a configurable simulation-oriented benchmark layer for underwater multi-robot racing. The proposed system adapts the repeatable, gate-based evaluation logic of autonomous racing to underwater vehicles while adding marine-specific elements such as acoustic guidance, depth constraints, currents, obstacles, and referee-controlled scoring. The central contribution is not a new simulator or a new controller, but a modular evaluation architecture in which controllers can be replaced while arena rules, task definitions, logs, and scoring remain fixed.

The current draft establishes the article structure, related work positioning, system architecture, environment design, evaluation protocol, and baseline status. The next required step is quantitative validation: repeated benchmark runs for acoustic-only and rule-based vision-assisted baselines on clean-gate, obstacle-gate, and current-gate tasks. Once those results are available, Marine Race Arena can be positioned

as a credible first step toward reproducible underwater racing benchmarks.

ACKNOWLEDGMENT

This section is intentionally left as a placeholder for funding, laboratory, and collaborator acknowledgments.

REFERENCES

- [1] M. O’Kelly, V. Sukhil, H. Abbas, J. Harkins, C. Kao, Y. V. Pant, R. Mangharam, D. Agarwal, M. Behl, P. Burgio, and M. Bertogna, “F1/10: An open-source autonomous cyber-physical platform,” *arXiv preprint arXiv:1901.08567*, 2019.
- [2] B. D. Evans, R. Trumpp, M. Caccamo, F. Jahncke, J. Betz, H. W. Jordaan, and H. A. Engelbrecht, “Unifying F1TENTH autonomous racing: Survey, methods and benchmarks,” *arXiv preprint arXiv:2402.18558*, 2024.
- [3] I. Charles, H. Maghsoumi, and Y. Fallah, “Advancing autonomous racing: A comprehensive survey of the RoboRacer (F1TENTH) platform,” *arXiv preprint arXiv:2506.15899*, 2025.
- [4] P. Foehn, D. Brescianini, E. Kaufmann, T. Cieslewski, M. Gehrig, M. Muglikar, and D. Scaramuzza, “Alphapilot: Autonomous drone racing,” *arXiv preprint arXiv:2005.12813*, 2020.
- [5] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, “Champion-level drone racing using deep reinforcement learning,” *Nature*, vol. 620, pp. 982–987, 2023.
- [6] S. A. Bahnam, R. Ferede, T. M. Blaha, A. E. Lang, E. Lucassen, Q. Missinne, A. E. C. Verraest, C. De Wagter, and G. C. H. E. de Croon, “MonoRace: Winning champion-level drone racing with robust monocular AI,” *arXiv preprint arXiv:2601.15222*, 2026.
- [7] M. Bosello, F. Pinzarrone, S. Kiade, D. Aguiari, Y. Keuter, A. AlShehhi, G. Caminati, K. L. Wong, K. S. Chou, J. Halepota, F. Alneyadi, J. Panerati, and G. Pau, “On your own: Pro-level autonomous drone racing in uninstrumented arenas,” *arXiv preprint arXiv:2510.13644*, 2025.
- [8] M. B. Azhari, D. Han, J. I. You, S. Park, and D. H. Shim, “Drift-corrected monocular VIO and perception-aware planning for autonomous drone racing,” *arXiv preprint arXiv:2512.20475*, 2025.
- [9] M. M. Zhang, W.-S. Choi, J. Herman, D. Davis, C. Vogt, M. McCarrin, Y. Vijay, D. Dutia, W. Lew, S. Peters, and B. Bingham, “DAVE aquatic virtual environment: Toward a general underwater robotics simulator,” *arXiv preprint arXiv:2209.02862*, 2022.
- [10] M. Grimaldi, P. Cieslak, E. Ochoa, V. Bharti, H. Rajani, I. Carlucho, M. Koskinopoulou, Y. R. Petillot, and N. Gracias, “Stonefish: Supporting machine learning research in marine robotics,” *arXiv preprint arXiv:2502.11887*, 2025.
- [11] A. Amer, O. Álvarez-Tuñón, H. I. Ugurlu, J. I. F. Sejersens, Y. Brodskiy, and E. Kayacan, “UNav-Sim: A visually realistic underwater robotics simulator and synthetic data-generation framework,” *arXiv preprint arXiv:2310.11927*, 2023.
- [12] S. Aldhaheri, Y. Hu, Y. Xie, P. Wu, D. Kanoulas, and Y. Liu, “Underwater robotic simulators review for autonomous system development,” *arXiv preprint arXiv:2504.06245*, 2025.
- [13] B. Romrell, A. Austin, B. Meyers, R. Anderson, C. Noh, and J. G. Mangelson, “A preview of HoloOcean 2.0,” *arXiv preprint arXiv:2510.06160*, 2025.
- [14] W. Denton, L. Chiavetta, M. Bryant, V. Shah, R. Zhu, R. Weerts, P. Xue, V. Chen, H. Le, M. Lin, A. Camacho, D. Council, E. Horowitz, J. Ong, M. Chu, and A. Pool, “Oogway: Designing, implementing, and testing an AUV for RoboSub 2023,” *arXiv preprint arXiv:2410.10900*, 2024.
- [15] W. Denton, M. Bryant, L. Chiavetta, V. Shah, R. Zhu, P. Xue, V. Chen, M. Lin, H. Le, A. Camacho, R. Galvez, N. Yang, N. Ren, T. Rose, M. Chu, A. Ergashev, S. Arya, K. Pieter, E. Horowitz, P. Zheng, and M. Kaarls, “Technical design review of duke robotics club’s oogway: An AUV for RoboSub 2024,” *arXiv preprint arXiv:2410.09684*, 2024.
- [16] Y.-W. Huang, T.-K. Chuang, N.-C. Lin, Y.-C. Hsiao, P.-W. Chen, C.-T. Hung, S.-H. Liu, H.-S. Chen, Y.-H. Hsieh, Y.-H. Huang, Y.-X. Chen, K.-L. Chen, Y.-J. Lan, C.-C. Hsu, C.-Y. Lin, J.-Y. Li, J.-T. Huang, Y.-J. Menn, S.-K. Lim, K.-B. Lua, C.-H. D. Tsai, and H.-C. Wang, “Team NCTU: Toward AI-driving for autonomous surface vehicles – from Duckietown to RobotX,” *arXiv preprint arXiv:1910.14540*, 2019.
- [17] A. Bhattacharya, A. Gandhi, L. Merkle, R. Tiwari, K. Warrior, S. Winata, A. Saba, K. Zhang, O. Kroemer, and S. Scherer, “Mission-level robustness with rapidly-deployed, autonomous aerial vehicles by carnegie mellon team tartan at MBZIRC 2020,” *arXiv preprint arXiv:2107.01507*, 2021.
- [18] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, “CARLA: An open urban driving simulator,” *arXiv preprint arXiv:1711.03938*, 2017.
- [19] J. Zilly, J. Tani, B. Considine, B. Mehta, A. F. Daniele, M. Diaz, G. Bernasconi, C. Ruch, J. Hakenberg, F. Golemo, A. K. Bowser, M. R. Walter, R. Hristov, S. Mallya, E. Frazzoli, A. Censi, and L. Paull, “The AI driving olympics at NeurIPS 2018,” *arXiv preprint arXiv:1903.02503*, 2019.
- [20] I. Zamora, N. G. Lopez, V. M. Vilches, and A. H. Cordero, “Extending the OpenAI Gym for robotics: A toolkit for reinforcement learning using ROS and Gazebo,” *arXiv preprint arXiv:1608.05742*, 2016.
- [21] A. Bedei, “HoloDroneCompetition: Marine race arena repository,” <https://github.com/AndreaBedei/HoloDroneCompetition>, 2026, accessed: 2026-06-24.